

Sistema de Gestão de Territórios e Publicações (GTP)

Documento 12 – Banco de Dados PostgreSQL

Projeto: Sistema de Gestão de Territórios e Publicações (GTP)

Versão: 1.0.0

Status: Em Elaboração

Data: Julho/2026

Autor: Fabio André Zatta

1 – Introdução

1.1 Objetivo

Este documento tem como objetivo definir a arquitetura física, a organização e as diretrizes de implementação do banco de dados do **Gestor de Territórios e Publicações (GTP)**, utilizando o **PostgreSQL 17** como Sistema Gerenciador de Banco de Dados (SGBD).

Enquanto o **Documento 11 – Modelo Entidade-Relacionamento (DER)** descreve a estrutura conceitual e lógica do domínio da aplicação, este documento detalha sua implementação física, estabelecendo padrões para tabelas, colunas, tipos de dados, índices, restrições, migrações, segurança, desempenho e administração do banco de dados.

Além disso, este documento servirá como referência para a integração do PostgreSQL com o backend desenvolvido em **Java 21**, **Spring Boot 3**, **Spring Data JPA**, **Hibernate 6** e **Flyway**, garantindo consistência entre a modelagem de dados e a implementação da aplicação.

1.2 Escopo

Este documento abrange todos os aspectos relacionados ao banco de dados do GTP, incluindo:

- arquitetura física do banco de dados;
- estrutura das tabelas;
- tipos de dados;
- relacionamentos físicos;
- índices e restrições;
- integridade referencial;
- convenções de nomenclatura;
- migrações com Flyway;
- estratégias de desempenho;
- administração do banco;
- segurança e controle de acesso;
- backup e recuperação;
- integração com a camada de persistência;
- boas práticas para evolução do esquema.

Não fazem parte do escopo deste documento:

- regras de negócio da aplicação;
- implementação da API REST;
- lógica dos casos de uso;
- componentes da interface do usuário;
- arquitetura do frontend.

Esses temas são tratados em seus respectivos documentos da arquitetura do projeto.

1.3 Público-Alvo

Este documento destina-se aos profissionais envolvidos no desenvolvimento, implantação, manutenção e evolução do GTP.

Entre os principais públicos estão:

- arquitetos de software;
- desenvolvedores backend;
- administradores de banco de dados (DBAs);
- engenheiros DevOps;
- analistas de sistemas;
- equipe de testes;
- equipe de infraestrutura;
- mantenedores do projeto.

A padronização aqui estabelecida permitirá que todos os envolvidos compartilhem uma visão única sobre a organização e operação do banco de dados.

1.4 Tecnologias Utilizadas

O banco de dados do GTP será construído utilizando tecnologias modernas e amplamente consolidadas no mercado.

Tecnologia	Finalidade
PostgreSQL 17	Sistema Gerenciador de Banco de Dados Relacional (SGBD).
Flyway	Controle de versão e migrações do esquema.
Spring Boot 3	Integração da aplicação com o banco de dados.
Spring Data JPA	Camada de acesso aos dados.
Hibernate 6	Implementação ORM e gerenciamento da persistência.
HikariCP	Pool de conexões JDBC.
Docker	Ambiente de execução do PostgreSQL.
Docker Compose	Orquestração dos serviços em ambiente local.
pgAdmin (opcional)	Administração e inspeção do banco de dados.

Essas tecnologias foram selecionadas por oferecerem alto desempenho, estabilidade, escalabilidade e ampla compatibilidade com o ecossistema Java.

1.5 Papel do PostgreSQL na Arquitetura do GTP

O PostgreSQL será responsável pelo armazenamento persistente de todas as informações gerenciadas pelo sistema.

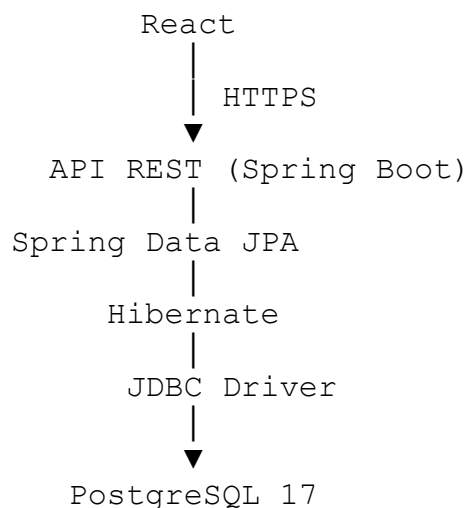
Entre suas responsabilidades destacam-se:

- armazenamento dos dados de negócio;
- garantia da integridade referencial;
- execução de transações ACID;
- gerenciamento de índices e consultas;
- suporte às migrações Flyway;
- controle de concorrência;
- auditoria e rastreabilidade;
- suporte às consultas analíticas e relatórios.

O banco de dados será considerado um componente central da arquitetura do sistema, interagindo diretamente com a camada de persistência implementada em Spring Data JPA.

1.6 Arquitetura Geral

A comunicação entre os componentes ocorrerá conforme o fluxo apresentado a seguir.



Toda comunicação entre a aplicação e o banco de dados ocorrerá exclusivamente por meio da camada de persistência, garantindo isolamento entre a lógica de negócio e o armazenamento físico dos dados.

1.7 Princípios Arquiteturais

A implementação do banco de dados seguirá os seguintes princípios:

- normalização adequada do modelo relacional;
- separação clara entre domínio e infraestrutura;
- integridade referencial rigorosa;
- versionamento completo do esquema;
- escalabilidade horizontal e vertical;
- otimização para consultas frequentes;
- segurança por padrão (*Security by Design*);
- facilidade de manutenção;
- observabilidade e rastreabilidade;
- compatibilidade com ambientes de desenvolvimento, homologação e produção.

Esses princípios orientam todas as decisões relacionadas ao projeto físico do banco de dados.

1.8 Convenções Gerais

Para manter uniformidade em todo o esquema do banco de dados, serão adotadas as seguintes convenções:

Tabelas

- nomes em letras minúsculas;
- utilização de *snake_case*;
- nomes no singular.

Exemplos:

usuario

perfil

territorio

publicador

designacao

Colunas

- letras minúsculas;
- *snake_case*;
- nomes descritivos.

Exemplos:

`data_criacao`

`data_atualizacao`

`congregacao_id`

Chaves Primárias

Todas as tabelas utilizarão:

`id`

como identificador principal.

Chaves Estrangeiras

Seguirão o padrão:

`<entidade>_id`

Exemplos:

`usuario_id`

`perfil_id`

`territorio_id`

`publicador_id`

Índices

Convenção:

`idx_<tabela>_<campo>`

Exemplos:

`idx_usuario_email`

`idx_publicador_nome`

`idx_territorio_numero`

Restrições

Convenção:

pk_<tabela>

fk_<origem>_<destino>

uk_<tabela>_<campo>

ck_<tabela>_<campo>

Esses padrões facilitam a identificação e manutenção dos objetos do banco.

1.9 Integração com os Demais Documentos

Este documento complementa diretamente os demais artefatos arquiteturais do GTP.

Documento	Relação
Documento 03 – Regras de Negócio	Origina diversas restrições implementadas no banco.
Documento 04 – Casos de Uso	Define as operações realizadas sobre os dados persistidos.
Documento 05 – Arquitetura do Sistema	Estabelece a organização geral dos módulos.
Documento 06 – Arquitetura Backend	Implementa a camada de persistência e acesso aos dados.
Documento 08 – Segurança	Define auditoria, autenticação e políticas de proteção dos dados.
Documento 09 – Docker	Disponibiliza o ambiente PostgreSQL em contêineres.
Documento 10 – API REST	Consome e manipula os dados armazenados no banco.
Documento 11 – Modelo Entidade-Relacionamento	Define o modelo conceitual e lógico implementado fisicamente neste documento.

Essa integração assegura consistência entre a documentação e a implementação do sistema.

1.10 Considerações Iniciais

O banco de dados do GTP foi concebido para ser um componente estratégico da arquitetura da aplicação, priorizando integridade, desempenho, segurança e

escalabilidade. A adoção do PostgreSQL 17, aliada ao ecossistema Spring Boot e Flyway, proporciona uma base sólida para suportar tanto as funcionalidades atuais quanto futuras evoluções do sistema.

Ao seguir as diretrizes estabelecidas neste documento, a equipe de desenvolvimento garantirá uma camada de persistência consistente, preparada para ambientes corporativos e alinhada às melhores práticas de engenharia de software.

2 – Arquitetura Geral do Banco de Dados

2.1 Objetivo

Este capítulo apresenta a arquitetura física do banco de dados do GTP, definindo a organização lógica e estrutural que sustentará a persistência de dados da aplicação.

São abordados:

- papel do PostgreSQL na arquitetura do sistema;
- organização física do banco de dados;
- ambientes de execução;
- estrutura por domínios;
- princípios de modelagem física;
- convenções arquiteturais;
- estratégias de escalabilidade e evolução.

O objetivo é garantir que a implementação do banco de dados seja consistente, segura, escalável e alinhada à arquitetura geral da solução.

2.2 Papel do PostgreSQL na Arquitetura do GTP

O PostgreSQL será o Sistema Gerenciador de Banco de Dados Relacional (SGBD) responsável pelo armazenamento persistente de todas as informações do GTP.

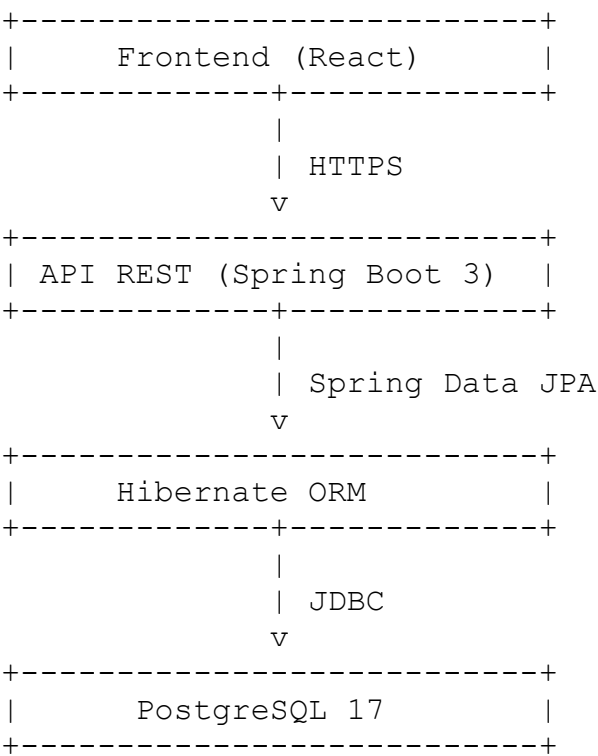
Entre suas principais responsabilidades destacam-se:

- armazenamento dos dados de negócio;
- gerenciamento das transações ACID;
- garantia da integridade referencial;
- execução de consultas SQL;
- gerenciamento de índices;
- suporte às migrações Flyway;
- controle de concorrência;
- armazenamento de auditoria;
- suporte a relatórios e indicadores.

O PostgreSQL atuará como a única fonte oficial de dados (*Single Source of Truth*), sendo acessado exclusivamente pela camada de persistência da aplicação.

2.3 Visão Geral da Arquitetura

A comunicação entre os componentes ocorrerá conforme o fluxo abaixo:



Essa arquitetura promove o desacoplamento entre a lógica de negócio e a persistência, facilitando testes, manutenção e evolução.

2.4 Organização Física do Banco

Inicialmente, o GTP utilizará um único banco de dados PostgreSQL, contendo todas as tabelas da aplicação.

Estrutura lógica:

```
Banco: gtp_db
├── Segurança
├── Administração
├── Congregações
├── Publicadores
├── Territórios
├── Designações
├── Estudos Bíblicos
├── Publicações
├── Notificações
├── Auditoria
└── Configurações
```

Embora todas as tabelas possam residir no schema padrão public na primeira versão, a arquitetura foi planejada para suportar a separação por schemas no futuro.

2.5 Organização por Domínios

A modelagem do banco segue a mesma divisão modular adotada na arquitetura do sistema.

Domínio	Responsabilidade
Segurança	Usuários, perfis, permissões, autenticação e tokens.
Administração	Configurações, parâmetros e auditoria.
Congregações	Dados das congregações.
Publicadores	Cadastro e organização dos publicadores.
Territórios	Territórios, quadras e endereços.
Designações	Distribuição e histórico de territórios.
Visitas	Registro das visitas realizadas.
Estudos Bíblicos	Controle de estudos bíblicos.
Publicações	Controle do acervo e distribuição.
Notificações	Comunicação com usuários.

Essa organização favorece a manutenção e reduz o acoplamento entre módulos.

2.6 Ambientes

O banco de dados será utilizado em três ambientes principais.

Ambiente	Finalidade
Desenvolvimento	Implementação e testes locais.
Homologação	Validação funcional e testes integrados.
Produção	Operação oficial do sistema.

Cada ambiente possuirá:

- banco independente;
- credenciais próprias;
- políticas específicas de backup;
- configuração de desempenho compatível com sua finalidade.

2.7 Estratégia de Schemas

Na primeira versão do GTP, será adotado o schema padrão:

```
public
```

Entretanto, a arquitetura permite futura segmentação em schemas especializados.

Exemplo:

```
public
```

```
security
```

```
audit
```

```
reports
```

```
integration
```

Essa abordagem poderá ser utilizada caso o sistema evolua para múltiplos módulos independentes ou exija maior isolamento entre áreas funcionais.

2.8 Convenções Arquiteturais

Para manter consistência no banco de dados, serão adotadas as seguintes convenções.

Tabelas

- nomes no singular;
- letras minúsculas;
- padrão snake_case.

Exemplos:

usuario

perfil

territorio

publicador

Colunas

- letras minúsculas;
- nomes descritivos;
- padrão snake_case.

Exemplos:

data_criacao

data_atualizacao

usuario_id

Chaves Primárias

Todas as tabelas utilizarão:

id BIGINT

como chave primária.

Chaves Estrangeiras

Seguirão o padrão:

<entidade>_id

Exemplos:

perfil_id

congregacao_id

`territorio_id`

Essas convenções simplificam a leitura dos scripts SQL e facilitam a integração com o ORM.

2.9 Princípios de Modelagem Física

A implementação física seguirá os seguintes princípios:

- normalização até, no mínimo, a Terceira Forma Normal (3FN);
- eliminação de redundâncias desnecessárias;
- uso consistente de chaves estrangeiras;
- restrições de integridade em nível de banco;
- utilização de índices para consultas frequentes;
- exclusão lógica para entidades de negócio;
- auditoria das operações críticas.

Esses princípios asseguram consistência e desempenho.

2.10 Estratégia de Escalabilidade

O modelo foi concebido para suportar crescimento gradual da aplicação.

As principais estratégias incluem:

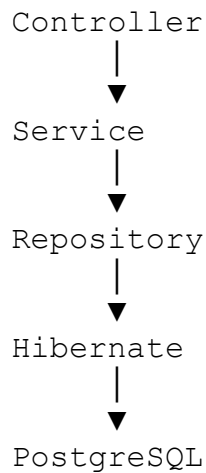
- indexação planejada;
- paginação em consultas;
- uso de EXPLAIN ANALYZE para otimização;
- particionamento futuro de tabelas volumosas (como auditoria e `historico_designacao`);
- materialized views para relatórios de alto custo;
- replicação de leitura em cenários de alta demanda.

Essas medidas permitem que o sistema evolua sem necessidade de reestruturações significativas.

2.11 Integração com a Camada de Persistência

O banco será acessado exclusivamente pela camada de persistência implementada em Spring Data JPA.

Fluxo de acesso:



Essa arquitetura garante encapsulamento do acesso aos dados e facilita a manutenção da aplicação.

2.12 Estratégia de Evolução

A evolução do banco será controlada por migrações Flyway.

Cada alteração estrutural deverá:

- possuir script próprio;
- ser versionada;
- ser executada automaticamente;
- manter compatibilidade entre ambientes.

Nenhuma alteração manual será realizada diretamente em produção.

2.13 Monitoramento

O banco de dados deverá ser monitorado continuamente para garantir estabilidade e desempenho.

Indicadores recomendados:

Indicador	Objetivo
Tempo médio de consultas	Identificar gargalos
Uso de CPU	Avaliar carga
Uso de memória	Detectar saturação
Tamanho do banco	Planejamento de capacidade

Indicador	Objetivo
Crescimento das tabelas	Definir estratégias de manutenção
Índices não utilizados	Otimização
Bloqueios	Identificar contenções

Essas métricas poderão ser coletadas por ferramentas de observabilidade integradas ao ambiente.

2.14 Relação com os Demais Documentos

Documento	Relação
Documento 05 – Arquitetura do Sistema	Define a organização geral da solução.
Documento 06 – Arquitetura Backend	Implementa a camada de persistência baseada nesta arquitetura.
Documento 08 – Segurança	Define políticas de proteção e auditoria dos dados.
Documento 09 – Docker	Disponibiliza o ambiente PostgreSQL para desenvolvimento e implantação.
Documento 10 – API REST	Consome e manipula os dados armazenados.
Documento 11 – Modelo Entidade-Relacionamento	Fornece o modelo conceitual e lógico implementado fisicamente neste documento.

2.15 Boas Práticas

Durante a implementação e manutenção do banco de dados deverão ser observadas as seguintes recomendações:

- manter sincronização entre DER, entidades JPA e migrações Flyway;
- evitar acesso direto ao banco fora da camada de persistência;
- utilizar transações para operações críticas;
- revisar periodicamente índices e consultas;
- documentar alterações estruturais;
- realizar backups antes de grandes migrações;
- monitorar continuamente o desempenho.

2.16 Conclusão do Capítulo

A arquitetura geral do banco de dados estabelece uma base sólida para a implementação física do GTP, definindo padrões de organização, convenções, estratégias de

crescimento e integração com a aplicação. O uso do PostgreSQL 17, aliado ao Spring Data JPA, Hibernate e Flyway, proporciona um ambiente robusto, escalável e preparado para suportar a evolução contínua da plataforma.

As decisões arquiteturais apresentadas neste capítulo garantem consistência entre o modelo conceitual, a implementação física e a camada de persistência, reduzindo riscos durante o desenvolvimento e a operação do sistema.

3 – Estrutura Física das Tabelas

3.1 Objetivo

Este capítulo descreve a estrutura física das tabelas que compõem o banco de dados do **Gestor de Territórios e Publicações (GTP)**.

São definidos:

- tabelas do sistema;
- colunas;
- tipos de dados;
- chaves primárias;
- chaves estrangeiras;
- restrições;
- índices;
- relacionamentos;
- finalidade de cada tabela.

Essa especificação servirá como referência para:

- implementação do PostgreSQL;
- criação das migrações Flyway;
- desenvolvimento das entidades JPA/Hibernate;
- documentação técnica do sistema.

3.2 Convenções Gerais

Todas as tabelas seguirão os seguintes padrões:

Nome das tabelas

- singular;
- letras minúsculas;
- snake_case.

Exemplos:

usuario

perfil

publicador

territorio

designacao

Chave Primária

Todas as tabelas utilizarão:

Campo Tipo

id BIGINT

Características:

- PRIMARY KEY;
- GENERATED BY DEFAULT AS IDENTITY;
- valor imutável.

Campos de Auditoria

As entidades de negócio compartilharão os seguintes campos:

Campo	Tipo
ativo	BOOLEAN
data_criacao	TIMESTAMP
data_atualizacao	TIMESTAMP
criado_por	BIGINT
atualizado_por	BIGINT
versao	BIGINT

Esses campos serão herdados da BaseEntity na camada JPA.

3.3 Organização das Tabelas

As tabelas estão organizadas conforme os domínios do sistema.

Domínio	Quantidade aproximada
Segurança	5
Administração	4
Congregações	2
Publicadores	5
Territórios	5
Designações	3
Estudos Bíblicos	2
Publicações	3
Notificações	2
Auditoria	2

Total previsto na versão 1.0:

30 a 35 tabelas

3.4 Domínio de Segurança

Tabela: perfil

Finalidade

Armazena os perfis de acesso do sistema.

Colunas

Campo	Tipo	Restrição
id	BIGINT	PK
nome	VARCHAR(100)	UNIQUE, NOT NULL
descricao	VARCHAR(255)	NULL

Índices

- uk_perfil_nome

Relacionamentos

- Perfil → Usuário (1:N)
- Perfil → Permissão (N:N)

Tabela: permissao

Finalidade

Lista todas as permissões disponíveis no sistema.

Campo	Tipo
id	BIGINT
codigo	VARCHAR(100)
descricao	VARCHAR(255)

Restrições:

- UNIQUE(codigo)

Tabela: perfil_permissao

Tabela associativa.

Campos:

Campo
perfil_id
permissao_id

PK composta:

perfil_id
permissao_id

Tabela: usuario

Representa os usuários autenticados.

Colunas principais

Campo	Tipo
id	BIGINT
nome	VARCHAR(150)
email	VARCHAR(150)

Campo	Tipo
senha	VARCHAR(255)
perfil_id	BIGINT
congregacao_id	BIGINT
ultimo_login	TIMESTAMP
ativo	BOOLEAN

Relacionamentos:

- Perfil
- Congregação
- Auditoria
- Refresh Token

Tabela: refresh_token

Armazena tokens JWT de renovação.

Campos:

- id
- usuario_id
- token
- expiracao
- revogado

3.5 Domínio Congregações

congregacao

Campos principais:

- id
- nome
- codigo
- endereco
- telefone
- cidade
- estado

Relacionamentos:

- Publicadores
- Usuários

- Territórios
- Configurações

configuracao

Armazena parâmetros específicos da congregação.

Campos:

- congregacao_id
 - chave
 - valor
-

3.6 Domínio Publicadores

publicador

Principais campos:

Campo

id

nome

sexo

telefone

email

data_batismo

data_nascimento

grupo_servico_id

congregacao_id

Relacionamentos:

- Grupo
- Congregação
- Designações
- Estudos Bíblicos
- Visitas

grupo_servico

Campos:

- id
- nome
- dirigente
- congregacao_id

estudo_biblico

Campos:

- publicador_id
- estudante
- endereco
- telefone
- observacoes

visita

Campos:

- publicador_id
- territorio_id
- data_visita
- resultado

3.7 Domínio Territórios

territorio

Campos:

- numero
- descricao
- status
- congregacao_id
- quadra_id

quadra

Campos:

- numero
- observacao

endereco

Campos:

- territorio_id
- logradouro
- numero
- bairro
- cidade
- cep

designacao

Campos:

- territorio_id
- publicador_id
- data_designacao
- data_devolucao
- status

historico_designacao

Campos:

- designacao_id
- data_evento
- usuario_id
- descricao

3.8 Domínio Publicações

publicacao

Campos:

- codigo
- titulo
- tipo
- idioma

estoque_publicacao

Campos:

- publicacao_id
- quantidade

movimentacao_publicacao

Campos:

- publicacao_id
- tipo
- quantidade
- data

3.9 Domínio Notificações

notificacao

Campos:

- usuario_id
- titulo
- mensagem
- lida
- data_envio

notificacao_usuario

Tabela opcional para múltiplos destinatários.

3.10 Domínio Auditoria

auditoria

Campos:

- usuario_id
- entidade
- operacao
- data_evento
- ip
- user_agent
- detalhes

log_sistema

Campos:

- nivel
- origem
- mensagem
- stacktrace
- data_evento

3.11 Campos Padronizados

Quase todas as tabelas utilizarão:

Campo	Tipo
ativo	BOOLEAN
data_criacao	TIMESTAMP
data_atualizacao	TIMESTAMP
criado_por	BIGINT
atualizado_por	BIGINT
versao	BIGINT

Esses campos garantem auditoria e controle de concorrência.

3.12 Convenções de Chaves

PK

pk_usuario

pk_publicador

pk_territorio

FK

fk_usuario_perfil

fk_publicador_congregacao

fk_territorio_congregacao

UNIQUE

uk_usuario_email

uk_perfil_nome

CHECK

ck_publicador_sexo

ck_designacao_status

3.13 Índices

Todos os seguintes campos possuirão índices:

- email
- codigo
- numero_territorio
- data_designacao
- status
- todas as FKs

Exemplo:

```
idx_usuario_email  
  
idx_publicador_nome  
  
idx_designacao_status
```

3.14 Relação com o JPA

Cada tabela possuirá:

- uma entidade JPA;
- um Repository;
- um Service;
- DTOs de entrada e saída;
- mapeamentos MapStruct;
- migração Flyway correspondente.

Essa organização garante alinhamento entre banco de dados e aplicação.

3.15 Relação com os Demais Documentos

Documento	Relação
Documento 06 – Arquitetura Backend	Implementa as entidades e repositórios correspondentes às tabelas.
Documento 10 – API REST	Expõe os dados persistidos por meio dos endpoints.
Documento 11 – DER	Define o modelo conceitual e lógico que fundamenta esta estrutura física.

3.16 Boas Práticas

Durante a implementação das tabelas deverão ser observadas as seguintes recomendações:

- utilizar nomes claros e padronizados;
- evitar redundância de informações;
- definir todas as restrições de integridade;
- indexar apenas colunas utilizadas em consultas frequentes;
- manter sincronização entre tabelas, entidades JPA e migrações Flyway;
- documentar alterações estruturais em novas migrações;
- preservar a compatibilidade entre versões do esquema.

3.17 Conclusão do Capítulo

A estrutura física das tabelas apresentada neste capítulo traduz o modelo conceitual do GTP em uma organização relacional pronta para implementação no PostgreSQL. A padronização de tabelas, colunas, chaves, restrições e índices garante consistência entre o banco de dados, a camada de persistência e a API REST, facilitando o desenvolvimento, a manutenção e a evolução da plataforma.

4 – Tipos de Dados, Domínios e Convenções

4.1 Objetivo

Este capítulo estabelece os padrões para utilização de tipos de dados, domínios reutilizáveis, convenções de nomenclatura e boas práticas na implementação física do banco de dados PostgreSQL do GTP.

Os objetivos são:

- padronizar a modelagem física;
- evitar inconsistências entre tabelas;
- facilitar a integração com Spring Data JPA e Hibernate;
- melhorar o desempenho das consultas;
- simplificar a manutenção e evolução do banco de dados.

4.2 Princípios Gerais

A definição dos tipos de dados seguirá os seguintes princípios:

- utilizar o menor tipo capaz de armazenar a informação;
- evitar desperdício de espaço em disco;
- privilegiar tipos nativos do PostgreSQL;
- garantir compatibilidade com Java 21 e Hibernate 6;
- utilizar restrições para preservar a integridade dos dados;
- evitar conversões desnecessárias entre aplicação e banco de dados.

4.3 Tipos de Dados Padronizados

A tabela a seguir apresenta os principais tipos de dados adotados no projeto.

Tipo PostgreSQL	Uso no GTP	Tipo Java
BIGINT	Identificadores (PK/FK)	Long
VARCHAR(n)	Textos curtos	String
TEXT	Textos longos	String
BOOLEAN	Valores lógicos	Boolean
INTEGER	Contadores e quantidades	Integer
NUMERIC(10,2)	Valores monetários	BigDecimal
DATE	Datas sem horário	LocalDate
TIMESTAMP WITH TIME ZONE	Data e hora	OffsetDateTime
TIME	Horários específicos	LocalTime
JSONB	Dados semiestruturados (quando necessário)	String ou objeto mapeado
BYTEA	Dados binários	byte[]

Esses tipos atendem às necessidades funcionais da aplicação e garantem portabilidade dentro do ecossistema Java.

4.4 Identificadores

Todas as entidades utilizarão identificadores numéricos.

Padrão

```
id BIGINT GENERATED BY DEFAULT AS IDENTITY
```

Características:

- chave primária;
- valor único;
- geração automática pelo PostgreSQL;
- imutável após criação.

A utilização de IDENTITY simplifica a integração com JPA e elimina a necessidade de sequências explícitas na maioria dos casos.

4.5 Tipos de Texto

VARCHAR

Utilizado para informações de tamanho conhecido.

Exemplos:

Campo Tamanho

nome 150

email 150

telefone 20

cidade 120

estado 2

CEP 9

código 50

TEXT

Utilizado para:

- observações;
- descrições extensas;
- mensagens;
- justificativas;
- conteúdo de auditoria.

4.6 Tipos Numéricos

INTEGER

Utilizado para:

- quantidades;
- prioridades;
- ordem de exibição;
- contadores.

NUMERIC

Utilizado para:

- valores monetários;
- estatísticas;
- indicadores.

Exemplo:

```
valor NUMERIC(10,2)
```

A escolha por NUMERIC evita perdas de precisão em cálculos financeiros.

4.7 Datas e Horários

A padronização das datas seguirá as recomendações modernas do PostgreSQL.

Tipo	Utilização
DATE	Datas sem horário (batismo, nascimento, vencimento).
TIMESTAMP WITH TIME ZONE	Auditoria, criação, atualização e eventos.
TIME	Horários específicos, quando aplicável.

Convenções

Campos de auditoria:

```
data_criacao  
data_atualizacao
```

Eventos:

```
data_designacao  
data_devolucao  
ultimo_login
```

O uso de `TIMESTAMP WITH TIME ZONE` garante consistência em ambientes distribuídos e facilita futuras integrações.

4.8 Valores Booleanos

Os campos booleanos representarão estados binários.

Exemplos:

```
ativo  
lido  
revogado  
administrador
```

Valores permitidos:

TRUE

FALSE

Não será utilizado NULL para estados booleanos, salvo quando houver justificativa técnica.

4.9 JSONB

O tipo JSONB será utilizado apenas quando houver necessidade de armazenar dados flexíveis ou configurações semiestruturadas.

Exemplos de uso:

- preferências de usuário;
- parâmetros de integração;
- configurações dinâmicas;
- metadados de notificações.

Seu uso deverá ser criterioso para evitar perda de normalização e dificuldades em consultas complexas.

4.10 Domínios Reutilizáveis

Sempre que apropriado, poderão ser definidos domínios PostgreSQL para padronizar tipos recorrentes.

Exemplos:

Domínio Base

email VARCHAR(150)

telefone VARCHAR(20)

cep VARCHAR(9)

codigo VARCHAR(50)

Essa abordagem centraliza validações e facilita futuras alterações.

4.11 Tipos Enumerados

Valores fechados poderão ser representados por ENUM ou por tabelas de domínio, conforme a necessidade de evolução.

Exemplos:

- sexo;
- status do território;
- status da designação;
- tipo de publicação;
- tipo de notificação.

Para conjuntos que possam crescer ou ser parametrizados pelos administradores, recomenda-se o uso de tabelas de domínio em vez de ENUM.

4.12 Convenções de Nomenclatura

Tabelas

- singular;
- letras minúsculas;
- snake_case.

Exemplos:

`usuario`

`perfil`

`publicador`

`territorio`

Colunas

- nomes descritivos;
- minúsculas;
- snake_case.

Exemplos:

`data_criacao`

`ultimo_login`

`congregacao_id`

4.13 Convenções para Restrições

Chaves Primárias

pk_usuario

pk_publicador

Chaves Estrangeiras

fk_usuario_perfil

fk_publicador_congregacao

Índices

idx_usuario_email

idx_publicador_nome

Restrições de Unicidade

uk_usuario_email

uk_perfil_nome

Restrições CHECK

ck_publicador_sexo

ck_designacao_status

Essa nomenclatura facilita a identificação dos objetos em logs, scripts e ferramentas de administração.

4.14 Valores Padrão

Sempre que adequado, serão definidos valores padrão (DEFAULT).

Exemplos:

Campo	Valor
ativo	TRUE
data_criacao	CURRENT_TIMESTAMP
versao	0
lido	FALSE

O uso de valores padrão reduz a necessidade de preenchimento manual pela aplicação.

4.15 Comentários no Banco de Dados

Todas as tabelas e colunas relevantes deverão possuir comentários (COMMENT ON) descrevendo sua finalidade.

Exemplo:

```
COMMENT ON TABLE publicador IS 'Armazena os dados dos  
publicadores da congregação.';
```

```
COMMENT ON COLUMN publicador.nome IS 'Nome completo do  
publicador.';
```

Essa prática melhora a documentação diretamente no banco de dados e facilita o uso de ferramentas de administração.

4.16 Compatibilidade com Spring Boot

Os tipos definidos neste documento foram selecionados para manter compatibilidade com:

- Java 21;
- Spring Boot 3;
- Spring Data JPA;
- Hibernate 6;
- Jakarta Persistence;
- Flyway.

Essa padronização reduz a necessidade de conversões e simplifica o mapeamento objeto-relacional.

4.17 Boas Práticas

Durante a implementação deverão ser observadas as seguintes recomendações:

- utilizar BIGINT para identificadores;
- preferir VARCHAR com tamanho definido a TEXT, quando possível;
- utilizar NUMERIC para valores monetários;
- padronizar datas com TIMESTAMP WITH TIME ZONE;
- evitar o uso indiscriminado de JSONB;
- definir DEFAULT para campos recorrentes;

- documentar tabelas e colunas com COMMENT ON;
- manter consistência entre PostgreSQL, entidades JPA e migrações Flyway.

4.18 Relação com os Demais Documentos

Documento	Relação
Documento 06 – Arquitetura Backend	Mapeia os tipos PostgreSQL para entidades Java e Hibernate.
Documento 09 – Docker	Define o ambiente de execução do PostgreSQL.
Documento 10 – API REST	Consome os dados persistidos conforme os tipos definidos neste capítulo.
Documento 11 – Modelo Entidade-Relacionamento	Fornece o modelo lógico que fundamenta a implementação física.

4.19 Conclusão do Capítulo

A padronização dos tipos de dados, domínios e convenções apresentada neste capítulo estabelece uma base consistente para a implementação física do banco de dados do GTP. A adoção de tipos apropriados, nomenclaturas uniformes e práticas recomendadas pelo PostgreSQL favorece a integridade dos dados, o desempenho das consultas e a integração com o ecossistema Spring.

Essas diretrizes reduzem ambiguidades, facilitam a manutenção e asseguram que a evolução do banco de dados ocorra de forma organizada e compatível com os demais componentes da arquitetura.

5 – Índices, Restrições e Integridade

5.1 Objetivo

Este capítulo define as estratégias para criação de índices, aplicação de restrições e garantia da integridade dos dados no banco de dados PostgreSQL do GTP.

Os objetivos são:

- assegurar a consistência das informações;
- preservar os relacionamentos entre entidades;
- otimizar consultas e operações de escrita;
- evitar duplicidade e dados inválidos;
- estabelecer padrões para criação de objetos de banco.

As diretrizes aqui apresentadas deverão ser aplicadas em todas as tabelas e migrações do sistema.

5.2 Princípios Gerais

A implementação seguirá os seguintes princípios:

- toda tabela deverá possuir chave primária;
- relacionamentos deverão utilizar chaves estrangeiras;
- dados obrigatórios utilizarão NOT NULL;
- unicidade será garantida por restrições UNIQUE;
- regras de domínio serão implementadas com CHECK;
- índices serão criados conforme o perfil de uso das consultas;
- a integridade deverá ser garantida prioritariamente pelo banco de dados.

Essa abordagem reduz inconsistências e complementa as validações realizadas pela aplicação.

5.3 Chaves Primárias (Primary Keys)

Cada tabela possuirá uma chave primária composta por um identificador único.

Padrão:

```
id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY
```

Convenção de nomenclatura

```
pk_<tabela>
```

Exemplos:

```
pk_usuario
```

```
pk_publicador
```

```
pk_territorio
```

```
pk_designacao
```

Diretrizes

- uma única chave primária por tabela;
- valor imutável;
- não reutilizar identificadores removidos;

- evitar chaves naturais como chave primária.

5.4 Chaves Estrangeiras (Foreign Keys)

As chaves estrangeiras garantem a integridade referencial entre as entidades.

Convenção

`fk_<tabela_origem>_<tabela_destino>`

Exemplos:

`fk_usuario_perfil`

`fk_publicador_congregacao`

`fk_designacao_territorio`

`fk_designacao_publicador`

Diretrizes

- toda relação obrigatória deverá possuir FOREIGN KEY;
- os tipos de dados da PK e da FK deverão ser idênticos;
- não serão permitidas referências órfãs;
- as relações deverão refletir fielmente o modelo do DER.

5.5 Restrições NOT NULL

Campos essenciais ao funcionamento do sistema deverão ser obrigatórios.

Exemplos:

Tabela	Campo
usuario	nome
usuario	email
usuario	senha
congregacao	nome
publicador	nome
territorio	numero

Essa restrição garante que informações críticas estejam sempre presentes.

5.6 Restrições UNIQUE

Campos que exigem unicidade utilizarão restrições específicas.

Exemplos

Campo	Justificativa
usuario.email	Um e-mail por usuário.
perfil.nome	Evita perfis duplicados.
congregacao.codigo	Identificador único da congregação.
territorio.numero	Número único dentro da congregação (em conjunto com a congregação, se aplicável).

Convenção

```
uk_<tabela>_<campo>
```

Exemplos:

```
uk_usuario_email
```

```
uk_perfil_nome
```

```
uk_congregacao_codigo
```

Quando a unicidade depender de mais de um campo, serão utilizadas restrições compostas.

5.7 Restrições CHECK

As restrições CHECK serão utilizadas para validar regras simples diretamente no banco de dados.

Exemplos

```
CHECK (sexo IN ('M', 'F'))
```

```
CHECK (quantidade >= 0)
```

```
CHECK (status IN ('ATIVO', 'INATIVO'))
```

```
CHECK (data_devolucao >= data_designacao)
```

Convenção

```
ck_<tabela>_<campo>
```

Exemplos:

`ck_publicador_sexo`

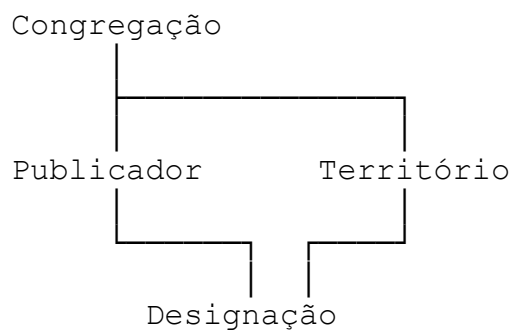
`ck_designacao_status`

`ck_estoque_quantidade`

5.8 Integridade Referencial

Todas as relações entre entidades serão protegidas por chaves estrangeiras.

Exemplo



Essa estrutura impede a criação de registros inconsistentes e assegura a coerência do domínio.

5.9 Políticas ON DELETE

A ação executada na exclusão de registros dependerá da natureza do relacionamento.

Política Utilização

RESTRICT Padrão para entidades de negócio.

CASCADE Tabelas associativas ou dependentes sem significado isolado.

SET NULL Relacionamentos opcionais.

Exemplos

- exclusão de um perfil será bloqueada se houver usuários vinculados (RESTRICT);
- exclusão de um perfil removerá automaticamente seus vínculos na tabela perfil_permissao (CASCADE);
- remoção de um usuário poderá manter registros históricos com o campo usuario_id nulo (SET NULL), quando apropriado.

5.10 Políticas ON UPDATE

Como as chaves primárias serão imutáveis, a política padrão será:

ON UPDATE RESTRICT

Essa decisão evita alterações que possam comprometer a integridade dos relacionamentos.

5.11 Estratégia de Indexação

Os índices serão criados para otimizar consultas frequentes, filtros, ordenações e junções.

Serão indexados:

- chaves primárias;
- chaves estrangeiras;
- campos utilizados em pesquisas;
- campos utilizados em ordenação;
- campos presentes em cláusulas WHERE;
- campos utilizados em JOIN.

5.12 Índices Simples

Exemplos:

```
idx_usuario_email
```

```
idx_publicador_nome
```

```
idx_congregacao_nome
```

```
idx_territorio_numero
```

Esses índices aceleram consultas sobre uma única coluna.

5.13 Índices Compostos

Quando consultas utilizarem múltiplos campos com frequência, serão adotados índices compostos.

Exemplos:

```
(congregacao_id, nome)

(congregacao_id, numero)

(publicador_id, data_designacao)

(status, data_designacao)
```

A ordem das colunas será definida conforme o perfil de acesso identificado em testes e monitoramento.

5.14 Índices Parciais

Para otimizar consultas sobre subconjuntos de dados, poderão ser utilizados índices parciais.

Exemplo:

```
CREATE INDEX idx_usuario_ativo

ON usuario (nome)

WHERE ativo = TRUE;
```

Essa estratégia reduz o tamanho dos índices e melhora o desempenho em consultas específicas.

5.15 Índices para Pesquisa Textual

Caso o sistema evolua para pesquisas avançadas em textos livres, poderão ser utilizados índices baseados em GIN e tsvector.

Exemplos de aplicação:

- pesquisa de publicadores;
- busca em observações;
- pesquisa de publicações;
- auditoria textual.

Essa funcionalidade será considerada em versões futuras.

5.16 Integridade Transacional

Todas as operações críticas serão executadas dentro de transações.

Exemplos:

- designação de territórios;
- devolução de territórios;
- movimentação de publicações;
- cadastro de usuários;
- atualização de configurações.

As transações seguirão as propriedades ACID:

- Atomicidade;
- Consistência;
- Isolamento;
- Durabilidade.

5.17 Exclusão Lógica (*Soft Delete*)

Entidades de negócio utilizarão exclusão lógica para preservar o histórico.

Campo padrão:

```
ativo BOOLEAN DEFAULT TRUE
```

A exclusão lógica:

- evita perda de informações;
- facilita auditorias;
- permite restauração de registros;
- preserva relacionamentos históricos.

A exclusão física será reservada para tabelas temporárias, de apoio ou associativas, quando apropriado.

5.18 Monitoramento de Índices

Os índices deverão ser revisados periodicamente para identificar:

- índices não utilizados;
- duplicidade de índices;
- fragmentação;
- impacto em operações de escrita;

- necessidade de novos índices.

Ferramentas como EXPLAIN ANALYZE e as visões estatísticas do PostgreSQL deverão ser utilizadas para apoiar essas análises.

5.19 Relação com Spring Data JPA

As restrições definidas no banco deverão refletir-se na camada de persistência.

Exemplos:

- `@Id;`
- `@GeneratedValue;`
- `@Column(nullable = false, unique = true);`
- `@ManyToOne;`
- `@OneToMany;`
- `@JoinColumn;`
- `@Version` (para controle de concorrência otimista).

Essa correspondência garante consistência entre a modelagem física e o código da aplicação.

5.20 Boas Práticas

Durante a implementação deverão ser observadas as seguintes recomendações:

- criar índices apenas quando justificados pelo uso;
- evitar excesso de índices, que pode degradar operações de escrita;
- definir restrições diretamente no banco, além das validações da aplicação;
- utilizar nomes padronizados para todos os objetos;
- revisar periodicamente o plano de execução das consultas;
- documentar toda alteração estrutural em migrações Flyway;
- validar impactos de desempenho antes de implantar novas estruturas em produção.

5.21 Relação com os Demais Documentos

Documento	Relação
Documento 06 – Arquitetura Backend	Implementa as restrições e relacionamentos por meio das entidades JPA/Hibernate.
Documento 09 – Docker	Disponibiliza o ambiente PostgreSQL para testes e implantação.

Documento	Relação
Documento 10 – API REST	Consome e manipula os dados respeitando as regras de integridade definidas neste capítulo.
Documento 11 – Modelo Entidade-Relacionamento	Define os relacionamentos lógicos materializados pelas chaves e restrições descritas aqui.

5.22 Conclusão do Capítulo

Os índices, restrições e mecanismos de integridade definidos neste capítulo constituem a base para a confiabilidade e o desempenho do banco de dados do GTP. A combinação de chaves primárias e estrangeiras, restrições de unicidade, validações em nível de banco e estratégias de indexação assegura que os dados permaneçam consistentes e que as consultas sejam executadas de forma eficiente.

Essas diretrizes complementam a modelagem física apresentada nos capítulos anteriores e fornecem os fundamentos necessários para a implementação das migrações Flyway, das entidades JPA/Hibernate e das operações da API REST.

6 – Views, Consultas e Objetos Auxiliares

6.1 Objetivo

Este capítulo define os objetos auxiliares do banco de dados PostgreSQL utilizados pelo GTP para apoiar consultas, relatórios, auditoria e automações.

São abordados:

- Views;
- Materialized Views;
- Sequences;
- Functions;
- Procedures;
- Triggers;
- Consultas reutilizáveis;
- Objetos de apoio à administração do banco.

A utilização desses recursos tem como objetivos:

- reduzir duplicação de consultas SQL;
- simplificar a camada de aplicação;
- melhorar o desempenho de relatórios;
- automatizar operações repetitivas;
- aumentar a consistência das informações.

6.2 Princípios Gerais

A utilização de objetos auxiliares seguirá os seguintes princípios:

- priorizar a lógica de negócio na aplicação (Spring Boot);
- utilizar objetos do banco apenas para responsabilidades técnicas ou de infraestrutura;
- documentar todos os objetos criados;
- versionar todos os scripts por meio do Flyway;
- manter nomenclatura padronizada.

6.3 Views

As *views* serão utilizadas para encapsular consultas frequentemente reutilizadas, facilitando a leitura e reduzindo a complexidade da camada de aplicação.

Objetivos

- simplificar consultas SQL;
- consolidar dados de múltiplas tabelas;
- padronizar relatórios;
- reduzir duplicação de código.

Convenção

`vw_<nome>`

Exemplos:

`vw_publicadores_ativos`

`vw_territorios_disponiveis`

`vw_designacoes_ativas`

`vw_publicacoes_estoque`

`vw_usuarios_administradores`

6.4 Views Operacionais

Algumas views serão destinadas ao apoio das operações do sistema.

Exemplo: Publicadores Ativos

Retorna:

- nome;
- grupo de serviço;
- congregação;
- telefone;
- situação.

Utilização:

- API REST;
- Dashboard;
- Relatórios.

Exemplo: Territórios Disponíveis

Retorna:

- número;
- bairro;
- congregação;
- status;
- última designação.

Utilização:

- distribuição de territórios;
- pesquisa;
- filtros.

6.5 Views para Relatórios

Views específicas serão utilizadas para alimentar os relatórios administrativos.

Exemplos:

```
vw_relatorio_territorios
```

```
vw_relatorio_designacoes
```

```
vw_relatorio_estudos
```

```
vw_relatorio_publicacoes
```

Essas views poderão consolidar dados de diversas tabelas, reduzindo a complexidade das consultas realizadas pela aplicação.

6.6 Views para Dashboards

Os indicadores exibidos no Dashboard utilizarão views específicas.

Exemplos:

```
vw_dashboard_publicadores
```

```
vw_dashboard_territorios
```

```
vw_dashboard_estudos
```

```
vw_dashboard_publicacoes
```

Essas views fornecerão informações como:

- total de publicadores;
- territórios disponíveis;
- territórios designados;
- estudos bíblicos ativos;
- movimentações recentes;
- indicadores de utilização.

6.7 Materialized Views

Para consultas analíticas de maior custo, poderão ser utilizadas *materialized views*.

Objetivos

- reduzir tempo de resposta;
- diminuir carga sobre o banco;
- melhorar desempenho de relatórios.

Exemplos:

```
mv_dashboard_indicadores
```

```
mv_relatorio_anual
```

```
mv_estatisticas_congregacao
```

Atualização

A atualização poderá ocorrer:

- manualmente;
- por agendamento;

- após eventos específicos;
- durante rotinas de manutenção.

A periodicidade dependerá da criticidade dos dados apresentados.

6.8 Sequences

Embora o PostgreSQL utilize IDENTITY como padrão para geração de identificadores, sequências dedicadas poderão ser adotadas quando houver necessidade de controle específico.

Exemplos:

- numeração de documentos;
- protocolos;
- identificadores externos.

Convenção:

`seq_<nome>`

Exemplos:

`seq_protocolo`

`seq_documento`

`seq_notificacao`

6.9 Functions

As funções armazenadas serão utilizadas para cálculos e operações reutilizáveis.

Diretrizes

- não conter regras de negócio complexas;
- retornar resultados determinísticos;
- ser reutilizáveis;
- possuir documentação.

Exemplos:

`fn_calcular_idade()`

`fn_nome_completo()`

```
fn_status_territorio()
```

```
fn_qtd_estudos_ativos()
```

Essas funções poderão ser utilizadas tanto pela aplicação quanto por consultas administrativas.

6.10 Procedures

Procedimentos armazenados serão empregados apenas quando houver necessidade de executar operações estruturadas diretamente no banco de dados.

Exemplos:

- manutenção periódica;
- arquivamento de registros;
- limpeza de dados temporários;
- atualização de estatísticas.

Convenção:

```
sp_<nome>
```

Exemplos:

```
sp_limpar_logs
```

```
sp_atualizar_estatisticas
```

```
sp_reindexar_banco
```

A lógica de negócio principal continuará concentrada na camada de aplicação.

6.11 Triggers

Os gatilhos (*triggers*) serão utilizados para automatizar operações técnicas.

Aplicações previstas:

- atualização automática de data_atualizacao;
- registro de auditoria;
- sincronização de informações derivadas;
- validações complementares;
- geração automática de históricos.

Convenção

`trg_<tabela>_<evento>`

Exemplos:

`trg_usuario_update`

`trg_designacao_insert`

`trg_publicador_delete`

6.12 Auditoria Automática

Triggers poderão registrar automaticamente alterações em tabelas críticas.

Eventos monitorados:

- INSERT;
- UPDATE;
- DELETE.

Informações registradas:

- usuário responsável;
- data e hora;
- operação executada;
- valores anteriores;
- valores atuais (quando aplicável).

Essa estratégia complementa o mecanismo de auditoria implementado pela aplicação.

6.13 Consultas Reutilizáveis

Consultas frequentemente utilizadas serão encapsuladas em views, funções ou repositórios especializados.

Exemplos:

- territórios disponíveis;
- publicadores por grupo;
- designações em aberto;
- estudos ativos;
- estatísticas da congregação.

Essa abordagem evita duplicação de SQL e facilita a manutenção.

6.14 Objetos de Administração

O banco poderá conter objetos auxiliares voltados à administração.

Exemplos:

- views de monitoramento;
- estatísticas de crescimento;
- consultas de diagnóstico;
- relatórios de utilização de índices.

Esses objetos apoiarão administradores de banco de dados e equipes DevOps.

6.15 Versionamento com Flyway

Todos os objetos auxiliares deverão ser criados e mantidos por meio de migrações Flyway.

Cada alteração deverá:

- possuir script próprio;
- ser versionada;
- ser documentada;
- manter compatibilidade entre ambientes.

Exemplo de nomenclatura:

```
V15__create_views_dashboard.sql
```

```
V16__create_functions.sql
```

```
V17__create_triggers.sql
```

6.16 Integração com Spring Boot

As views e funções poderão ser consumidas pela aplicação por meio de:

- Spring Data JPA;
- consultas nativas (`@Query(nativeQuery = true)`);
- projeções (`Projection`);
- DTOs;

- repositórios especializados.

Sempre que possível, a aplicação utilizará entidades e repositórios padrão, reservando consultas nativas para cenários específicos.

6.17 Desempenho

A criação de objetos auxiliares deverá considerar:

- custo das consultas;
- frequência de atualização dos dados;
- impacto sobre operações de escrita;
- necessidade de índices adicionais;
- utilização de EXPLAIN ANALYZE.

Objetos que apresentarem baixa utilização deverão ser revisados ou removidos.

6.18 Segurança

Acesso a views e funções poderá ser controlado por permissões específicas.

Exemplos:

- usuários administrativos;
- usuários somente leitura;
- processos automatizados;
- integrações externas.

Essa estratégia reforça o princípio do menor privilégio.

6.19 Boas Práticas

Durante a criação de objetos auxiliares deverão ser observadas as seguintes recomendações:

- utilizar nomenclatura padronizada;
- documentar todas as views, funções e triggers;
- evitar lógica de negócio complexa no banco de dados;
- priorizar consultas reutilizáveis;
- utilizar *materialized views* apenas quando houver ganho comprovado de desempenho;
- monitorar o impacto de objetos auxiliares nas operações do banco;
- versionar todas as alterações com Flyway.

6.20 Relação com os Demais Documentos

Documento	Relação
Documento 06 – Arquitetura Backend	Consome views, funções e consultas por meio da camada de persistência.
Documento 08 – Segurança	Define políticas de acesso a objetos do banco e mecanismos de auditoria.
Documento 09 – Docker	Disponibiliza o ambiente PostgreSQL contendo todos os objetos auxiliares.
Documento 10 – API REST	Utiliza views e consultas para geração de relatórios e indicadores.
Documento 11 – Modelo Entidade-Relacionamento	Fornece a base estrutural utilizada pelas views e consultas.

6.21 Conclusão do Capítulo

As *views*, *materialized views*, funções, procedimentos e gatilhos descritos neste capítulo ampliam as capacidades do PostgreSQL sem comprometer a separação de responsabilidades entre o banco de dados e a aplicação. Ao concentrar no banco apenas funcionalidades técnicas e de apoio, o GTP preserva a lógica de negócio na camada de serviços do Spring Boot, mantendo uma arquitetura limpa, modular e de fácil evolução.

Além disso, a padronização de nomenclatura, o versionamento por Flyway e a integração com a camada de persistência asseguram que esses objetos possam ser mantidos de forma consistente em todos os ambientes do sistema.

7 – Migrações Flyway

7.1 Objetivo

Este capítulo estabelece as diretrizes para utilização do **Flyway** no gerenciamento das migrações do banco de dados PostgreSQL do GTP.

São definidos:

- estratégia de versionamento do esquema;
- organização dos scripts SQL;
- convenções de nomenclatura;
- controle de execução das migrações;
- evolução do modelo de dados;
- migrações estruturais e de dados;
- boas práticas de implantação.

O objetivo é garantir que todas as alterações no banco de dados sejam controladas, auditáveis e reproduzíveis em qualquer ambiente.

7.2 O Papel do Flyway

O Flyway será a ferramenta oficial para gerenciamento de versões do banco de dados.

Suas principais responsabilidades são:

- versionar o esquema do banco;
- aplicar migrações automaticamente;
- controlar o histórico de alterações;
- impedir execução duplicada de scripts;
- validar a integridade das migrações;
- sincronizar os ambientes de desenvolvimento, homologação e produção.

O Flyway utilizará a tabela de controle flyway_schema_history, criada automaticamente, para registrar todas as migrações executadas.

7.3 Estrutura de Diretórios

Os scripts de migração serão organizados no projeto backend conforme a estrutura abaixo:

```
src/
├── main/
│   └── resources/
│       └── db/
│           └── migration/
│               ├── V1__initial_schema.sql
│               ├── V2__create_security_tables.sql
│               ├── V3__create_congregacao_tables.sql
│               ├── V4__create_publicador_tables.sql
│               ├── V5__create_territorio_tables.sql
│               ├── V6__create_designacao_tables.sql
│               ├── V7__create_publicacao_tables.sql
│               ├── V8__create_auditoria_tables.sql
│               └── ...
```

Essa organização facilita a localização e manutenção das migrações.

7.4 Convenção de Nomenclatura

Todos os scripts seguirão o padrão oficial do Flyway:

V<versão>__<descrição>.sql

Exemplos:

V1__initial_schema.sql

V2__create_security_tables.sql

V3__create_congregacao_tables.sql

V4__create_publicador_tables.sql

V5__create_territorio_tables.sql

V6__create_designacao_tables.sql

V7__create_publicacao_tables.sql

V8__create_views.sql

V9__create_indexes.sql

V10__seed_initial_data.sql

Regras:

- utilizar letras minúsculas na descrição;
- separar palavras com _;
- descrição clara e objetiva;
- numeração sequencial e sem reutilização.

7.5 Tipos de Migração

O GTP utilizará principalmente migrações versionadas (Versioned Migrations), que representam alterações permanentes no esquema do banco.

As migrações poderão incluir:

- criação de tabelas;
- alteração de colunas;
- criação de índices;
- inclusão de restrições;
- criação de views;

- criação de funções;
- criação de triggers;
- inserção de dados essenciais.

Migrações repetíveis (Repeatable Migrations) poderão ser utilizadas para objetos como views ou funções que precisem ser recriados integralmente após alterações.

7.6 Estratégia de Evolução do Esquema

Cada mudança estrutural deverá corresponder a uma nova migração.

Exemplos:

Alteração	Nova migração
Criar tabela usuario	V2__create_usuario_table.sql
Adicionar coluna telefone	V11__add_telefone_to_usuario.sql
Criar índice em email	V12__create_usuario_email_index.sql
Criar view de dashboard	V13__create_dashboard_views.sql

Alterações em scripts já executados não são permitidas, pois comprometem a rastreabilidade entre ambientes.

7.7 Migrações de Dados

Além das alterações estruturais, o Flyway será utilizado para inserir dados obrigatórios ao funcionamento do sistema.

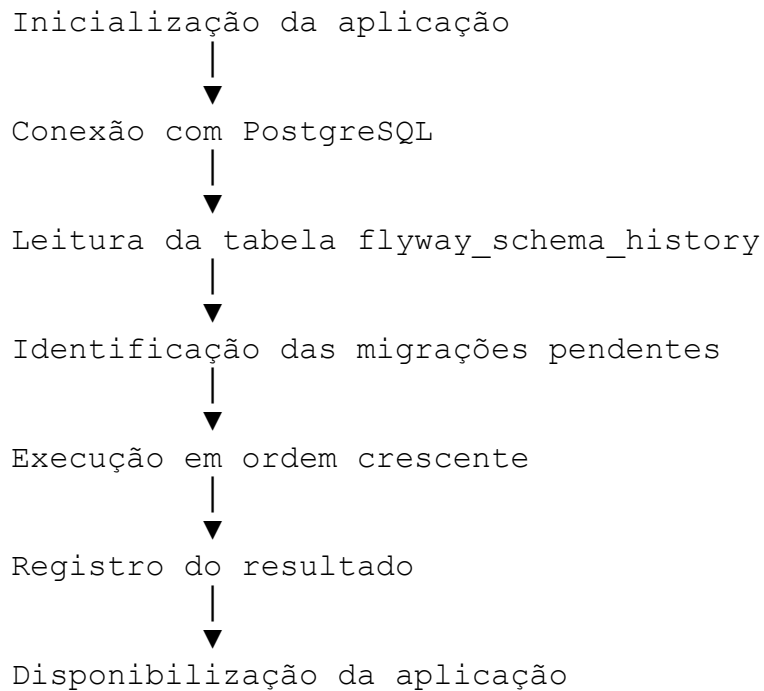
Exemplos:

- perfis padrão;
- permissões iniciais;
- configurações globais;
- parâmetros do sistema;
- tipos de publicações;
- estados e códigos padronizados.

Essas migrações deverão ser idempotentes sempre que possível, evitando inserções duplicadas.

7.8 Controle de Execução

Ao iniciar a aplicação, o Flyway executará o seguinte fluxo:



Esse processo garante que o banco esteja sempre compatível com a versão da aplicação.

7.9 Ambientes

As mesmas migrações serão utilizadas em todos os ambientes.

Ambiente	Uso
Desenvolvimento	Criação e testes das migrações.
Homologação	Validação integrada.
Produção	Implantação oficial.

Não serão mantidos scripts diferentes para cada ambiente, exceto quando houver necessidade técnica devidamente documentada.

7.10 Tratamento de Falhas

Caso uma migração apresente erro:

- a execução será interrompida;
- a transação será revertida (quando suportado pelo comando executado);

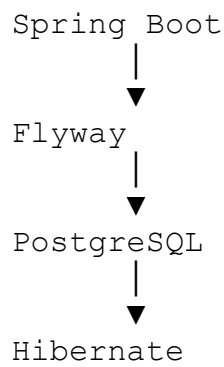
- o banco permanecerá consistente;
- a correção será realizada em um novo script de migração, sem alterar versões já aplicadas.

Após a correção, a equipe poderá utilizar os recursos do Flyway para validar e reparar o histórico, quando necessário.

7.11 Integração com Spring Boot

O Flyway será integrado ao ciclo de inicialização da aplicação Spring Boot.

Fluxo simplificado:



As migrações serão executadas antes da inicialização do Hibernate, garantindo que o esquema do banco esteja atualizado.

7.12 Relação com o Hibernate

O GTP adotará a seguinte estratégia:

- Flyway será o responsável exclusivo pela criação e evolução do esquema do banco;
- Hibernate será utilizado apenas para o mapeamento objeto-relacional.

Em produção, recomenda-se configurar:

`spring.jpa.hibernate.ddl-auto=validate`

Essa configuração faz com que o Hibernate valide o esquema existente, sem criar ou alterar tabelas automaticamente.

7.13 Organização por Domínio

À medida que o projeto evoluir, as migrações poderão ser agrupadas por módulos funcionais.

Exemplo de sequência:

Versão Conteúdo

V1	Estrutura inicial
V2	Segurança
V3	Congregações
V4	Publicadores
V5	Territórios
V6	Designações
V7	Estudos Bíblicos
V8	Publicações
V9	Auditoria
V10	Configurações

Essa organização facilita a rastreabilidade das alterações.

7.14 Controle de Qualidade

Antes da implantação, cada migração deverá ser submetida às seguintes verificações:

- sintaxe SQL válida;
- execução em banco limpo;
- execução em banco já existente;
- compatibilidade com PostgreSQL 17;
- impacto sobre índices e desempenho;
- integridade referencial preservada;
- compatibilidade com entidades JPA.

Essas validações reduzem riscos durante a implantação.

7.15 Boas Práticas

Durante o desenvolvimento das migrações deverão ser observadas as seguintes recomendações:

- criar uma migração para cada alteração lógica;
- utilizar descrições claras e objetivas;

- nunca modificar scripts já executados em ambientes compartilhados;
- evitar múltiplas alterações não relacionadas em um único arquivo;
- documentar mudanças relevantes;
- manter scripts pequenos e focados;
- revisar impactos antes da implantação;
- testar todas as migrações em ambiente de desenvolvimento.

7.16 Relação com os Demais Documentos

Documento	Relação
Documento 06 – Arquitetura Backend	Define a integração entre Flyway, Hibernate e Spring Data JPA.
Documento 09 – Docker	Configura o ambiente PostgreSQL utilizado para execução das migrações.
Documento 10 – API REST	Depende do esquema atualizado para funcionamento dos endpoints.
Documento 11 – Modelo Entidade-Relacionamento	Serve de base para a criação das migrações estruturais.
Documento 12 – Banco de Dados PostgreSQL	Detalha a implementação física versionada pelo Flyway.

7.17 Considerações sobre Evolução Contínua

O banco de dados do GTP foi projetado para evoluir continuamente. O uso do Flyway garante que cada alteração seja registrada de forma histórica, permitindo reproduzir o esquema completo em qualquer ambiente e facilitando auditorias, revisões e implantações.

Essa estratégia favorece práticas de Integração Contínua (CI) e Entrega Contínua (CD), reduzindo riscos associados à evolução do banco de dados.

7.18 Conclusão do Capítulo

A adoção do Flyway como ferramenta oficial de migração estabelece um processo seguro e padronizado para evolução do banco de dados do GTP. O versionamento das alterações, aliado à integração com Spring Boot e Hibernate, garante que todos os ambientes permaneçam sincronizados e que a estrutura do banco evolua de maneira previsível, auditável e alinhada às boas práticas de engenharia de software.

8 – Administração do Banco

8.1 Objetivo

Este capítulo define as práticas e procedimentos para administração do banco de dados PostgreSQL utilizado pelo GTP.

São abordados:

- administração do ambiente PostgreSQL;
- gerenciamento de usuários e permissões;
- manutenção preventiva e corretiva;
- monitoramento operacional;
- gerenciamento de conexões;
- otimização de desempenho;
- planejamento de capacidade;
- políticas de backup e recuperação.

Essas diretrizes asseguram que o banco de dados permaneça confiável, seguro e preparado para suportar o crescimento da aplicação.

8.2 Princípios de Administração

A administração do banco seguirá os seguintes princípios:

- disponibilidade contínua;
- integridade dos dados;
- segurança por padrão (*Security by Default*);
- automação de tarefas recorrentes;
- monitoramento proativo;
- rastreabilidade das alterações;
- escalabilidade planejada;
- mínima intervenção manual em produção.

8.3 Ambientes

O PostgreSQL será mantido em ambientes independentes.

Ambiente	Finalidade
Desenvolvimento	Implementação e testes locais.
Homologação	Validação integrada antes da produção.
Produção	Operação oficial do sistema.

Cada ambiente possuirá:

- banco de dados próprio;
- credenciais exclusivas;
- configuração específica;
- rotinas independentes de backup;
- monitoramento próprio.

8.4 Administração de Usuários

O acesso ao banco será realizado por usuários distintos conforme a finalidade.

Usuário	Finalidade
postgres	Administração do SGBD.
gtp_app	Aplicação Spring Boot.
gtp_migration	Execução das migrações Flyway.
gtp_readonly	Consultas e relatórios, quando necessário.

Diretrizes

- não utilizar o usuário postgres pela aplicação;
- conceder apenas os privilégios necessários;
- utilizar senhas fortes e exclusivas;
- revisar permissões periodicamente.

8.5 Controle de Permissões

O princípio do menor privilégio será adotado.

Exemplo de permissões:

Perfil	Permissões
Aplicação	SELECT, INSERT, UPDATE, DELETE nas tabelas necessárias.
Flyway	Alterações de esquema (CREATE, ALTER, DROP) durante as migrações.
Leitura	Apenas SELECT em tabelas e views autorizadas.

Administração Controle completo do banco de dados.

Essa separação reduz riscos de alterações acidentais ou indevidas.

8.6 Gerenciamento de Conexões

As conexões serão gerenciadas pelo **HikariCP**, configurado na aplicação Spring Boot.

Diretrizes

- limitar o número máximo de conexões;
- configurar tempo de espera adequado;
- monitorar conexões ativas e ociosas;
- evitar conexões abertas desnecessariamente;
- reutilizar conexões por meio do pool.

A configuração será ajustada conforme a capacidade do ambiente e o perfil de carga.

8.7 Manutenção Preventiva

Rotinas periódicas de manutenção deverão ser executadas para preservar o desempenho do banco.

VACUUM

Responsável por recuperar espaço e manter a eficiência do armazenamento.

Utilizações:

- remoção de registros obsoletos;
- atualização de estatísticas internas;
- redução da fragmentação.

ANALYZE

Atualiza as estatísticas utilizadas pelo otimizador de consultas.

Benefícios:

- planos de execução mais eficientes;
- melhor utilização de índices;
- redução do tempo de resposta.

REINDEX

Deverá ser utilizado quando houver indícios de degradação dos índices.

Situações comuns:

- crescimento excessivo;
- fragmentação;
- degradação de desempenho.

8.8 Monitoramento Operacional

O ambiente deverá ser monitorado continuamente.

Indicadores recomendados:

Indicador	Objetivo
Uso de CPU	Avaliar carga do servidor.
Uso de memória	Identificar saturação.

Indicador	Objetivo
Espaço em disco	Planejar expansão.
Número de conexões	Detectar gargalos.
Consultas lentas	Identificar oportunidades de otimização.
Locks	Detectar contenções.
Crescimento das tabelas	Planejamento de capacidade.
Uso de índices	Avaliar eficiência das consultas.

Ferramentas como **Prometheus**, **Grafana** e **pgAdmin** poderão ser utilizadas para acompanhamento dessas métricas.

8.9 Monitoramento de Consultas

Consultas com alto custo deverão ser analisadas periodicamente.

Ferramentas recomendadas:

- EXPLAIN;
- EXPLAIN ANALYZE;
- pg_stat_statements;
- estatísticas do PostgreSQL.

Objetivos:

- identificar consultas lentas;
- avaliar planos de execução;
- revisar índices;
- otimizar filtros e junções.

8.10 Planejamento de Capacidade

O crescimento do banco será acompanhado continuamente.

Aspectos monitorados:

- volume de dados;

- crescimento mensal;
- utilização de armazenamento;
- evolução do número de registros;
- necessidade de particionamento;
- projeção de capacidade.

A análise periódica permitirá antecipar necessidades de expansão.

8.11 Políticas de Backup

O banco de dados deverá possuir uma política formal de backup.

Tipos de backup:

Tipo	Frequência
Completo	Diário
Incremental	Conforme necessidade operacional.
Backup antes de migrações críticas	Obrigatório
Backup antes de atualizações da aplicação	Obrigatório

Os backups deverão ser testados regularmente para garantir sua recuperação.

8.12 Recuperação de Desastres

O plano de recuperação deverá contemplar:

- restauração de backups;
- validação da integridade dos dados;
- reexecução de migrações Flyway, quando aplicável;
- recuperação do ambiente Docker;
- retorno seguro à operação.

Os procedimentos deverão ser documentados e periodicamente testados.

8.13 Administração de Logs

Os logs do PostgreSQL deverão ser configurados para registrar:

- erros;
- consultas lentas;
- conexões;
- autenticações;
- bloqueios;
- falhas de integridade.

Os logs deverão possuir política de retenção e rotação para evitar crescimento descontrolado.

8.14 Atualizações do PostgreSQL

A atualização do PostgreSQL seguirá um processo controlado.

Etapas recomendadas:

1. validar compatibilidade em desenvolvimento;
2. testar em homologação;
3. realizar backup completo;
4. executar atualização;
5. validar aplicação e migrações;
6. monitorar estabilidade após a implantação.

Atualizações nunca deverão ser realizadas diretamente em produção sem testes prévios.

8.15 Administração com Docker

Em ambientes containerizados, o PostgreSQL será administrado por meio do Docker Compose.

Responsabilidades:

- inicialização do serviço;

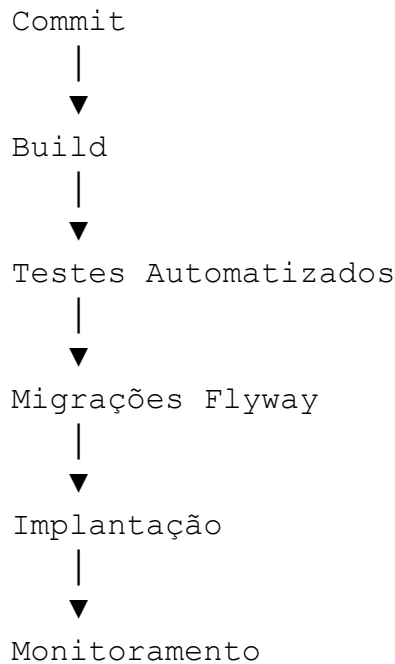
- gerenciamento de volumes persistentes;
- configuração de redes;
- definição de variáveis de ambiente;
- monitoramento dos contêineres.

Os volumes deverão ser persistentes para garantir a preservação dos dados em reinicializações.

8.16 Integração com DevOps

A administração do banco deverá estar integrada ao pipeline de DevOps.

Fluxo recomendado:



Essa integração reduz falhas manuais e aumenta a confiabilidade das implantações.

8.17 Boas Práticas

Durante a administração do banco deverão ser observadas as seguintes recomendações:

- utilizar usuários específicos para cada finalidade;
- aplicar o princípio do menor privilégio;

- monitorar continuamente desempenho e capacidade;
- executar rotinas de manutenção periodicamente;
- validar backups e restaurações;
- documentar procedimentos operacionais;
- automatizar tarefas recorrentes;
- evitar alterações manuais em produção.

8.18 Relação com os Demais Documentos

Documento	Relação
Documento 06 – Arquitetura Backend	Define a integração da aplicação com o banco de dados.
Documento 08 – Segurança	Estabelece políticas de autenticação, autorização e proteção dos dados.
Documento 09 – Docker	Descreve a infraestrutura containerizada utilizada pelo PostgreSQL.
Documento 10 – API REST	Consome os dados administrados pelo banco.
Documento 12 – Banco de Dados PostgreSQL	Complementa as diretrizes operacionais da plataforma de persistência.

8.19 Considerações Finais

A administração eficiente do PostgreSQL é essencial para garantir a continuidade dos serviços do GTP. A combinação de monitoramento contínuo, manutenção preventiva, políticas de backup, controle de acesso e integração com práticas de DevOps assegura um ambiente robusto, seguro e preparado para suportar a evolução do sistema.

Ao seguir as diretrizes estabelecidas neste capítulo, a equipe técnica poderá administrar o banco de dados de forma padronizada, reduzindo riscos operacionais e mantendo elevados níveis de disponibilidade e desempenho.

9 – Segurança do Banco de Dados

9.1 Objetivo

Este capítulo estabelece as políticas e mecanismos de segurança aplicados ao banco de dados PostgreSQL do GTP.

São abordados:

- autenticação e autorização;
- gerenciamento de usuários e papéis (*roles*);
- proteção das credenciais;
- criptografia em trânsito e em repouso;
- auditoria;
- monitoramento de acessos;
- conformidade com a LGPD;
- endurecimento (*hardening*) do ambiente;
- resposta a incidentes.

O objetivo é garantir a confidencialidade, integridade, disponibilidade e rastreabilidade das informações armazenadas.

9.2 Princípios de Segurança

A arquitetura de segurança do banco será baseada nos seguintes princípios:

- Princípio do Menor Privilégio (*Least Privilege*);
- Defesa em Profundidade (*Defense in Depth*);
- Segregação de Funções (*Separation of Duties*);
- Segurança por Padrão (*Security by Default*);
- Criptografia sempre que aplicável;
- Auditoria de operações críticas;
- Atualizações regulares do ambiente;
- Monitoramento contínuo.

Esses princípios orientam todas as decisões relacionadas à administração e proteção do banco de dados.

9.3 Autenticação

O acesso ao PostgreSQL será realizado exclusivamente por usuários autenticados.

Diretrizes

- autenticação obrigatória para todas as conexões;
- senhas fortes e exclusivas para cada usuário;
- proibição de contas compartilhadas;
- bloqueio de autenticação anônima;
- renovação periódica de credenciais em ambientes críticos.

Em produção, recomenda-se utilizar autenticação integrada a mecanismos seguros de gerenciamento de identidades quando disponível.

9.4 Gerenciamento de Usuários e Papéis

O PostgreSQL utilizará *roles* distintas para separar responsabilidades.

Papel	Finalidade
postgres	Administração do SGBD.
gtp_app	Acesso da aplicação Spring Boot.
gtp_migration	Execução das migrações Flyway.
gtp_readonly	Consultas e relatórios.
gtp_backup	Execução de rotinas de backup.

Diretrizes

- evitar utilização do superusuário pela aplicação;
- conceder apenas os privilégios estritamente necessários;
- revisar permissões periodicamente;
- remover usuários e papéis obsoletos.

9.5 Controle de Acesso

O acesso aos objetos do banco será controlado por permissões específicas.

Exemplos:

Objeto	Permissão
--------	-----------

Tabelas operacionais	SELECT, INSERT, UPDATE, DELETE para gtp_app.
----------------------	--

Views de relatórios	SELECT para usuários autorizados.
---------------------	-----------------------------------

Migrações	CREATE, ALTER, DROP apenas para gtp_migration.
-----------	--

Administração	Acesso restrito ao administrador do banco.
---------------	--

Todas as permissões deverão ser concedidas explicitamente, evitando privilégios excessivos.

9.6 Proteção das Credenciais

As credenciais de acesso ao banco não deverão ser armazenadas em código-fonte.

Diretrizes

- utilizar variáveis de ambiente;
- empregar arquivos de configuração protegidos;
- integrar com gerenciadores de segredos em ambientes produtivos;
- restringir o acesso às credenciais.

Exemplo de configuração no Spring Boot:

```
spring.datasource.url=jdbc:postgresql://localhost:5432/gtp_db
spring.datasource.username=${DB_USERNAME}
spring.datasource.password=${DB_PASSWORD}
```

Essa abordagem reduz o risco de exposição acidental de informações sensíveis.

9.7 Criptografia em Trânsito

Todas as conexões entre a aplicação e o PostgreSQL deverão utilizar canais seguros sempre que o banco estiver acessível por rede.

Recomendações

- utilizar TLS/SSL;
- validar certificados digitais;
- impedir conexões não criptografadas em produção;
- proteger comunicações entre contêineres quando distribuídos em diferentes hosts.

A criptografia em trânsito evita interceptação de credenciais e dados.

9.8 Criptografia em Repouso

Sempre que possível, os dados armazenados deverão ser protegidos por mecanismos de criptografia em repouso.

Exemplos:

- criptografia de volumes de disco;
- criptografia fornecida pelo provedor de nuvem;
- proteção de backups.

Para informações extremamente sensíveis, poderá ser considerada criptografia em nível de aplicação antes da persistência.

9.9 Proteção de Dados Sensíveis

Algumas informações exigem tratamento diferenciado.

Exemplos:

- senhas;
- tokens de autenticação;
- chaves de integração;
- dados pessoais protegidos pela LGPD.

Diretrizes

- senhas armazenadas apenas como hash seguro (ex.: BCrypt, Argon2), nunca em texto plano;
- tokens com prazo de validade e revogação;
- dados sensíveis acessíveis apenas por perfis autorizados;
- mascaramento de informações em consultas administrativas, quando aplicável.

9.10 Auditoria

Todas as operações críticas deverão ser registradas.

Eventos auditáveis:

- autenticações;
- falhas de login;
- criação e remoção de usuários;
- alterações estruturais;
- modificações em dados críticos;
- execução de migrações.

As informações registradas incluirão:

- usuário;
- data e hora;
- endereço IP (quando disponível);
- operação realizada;
- objeto afetado.

9.11 Monitoramento de Segurança

O ambiente deverá ser monitorado continuamente para identificar comportamentos suspeitos.

Eventos monitorados:

- tentativas repetidas de autenticação;
- acessos fora do horário esperado;
- aumento incomum de consultas;
- alterações em permissões;
- falhas de integridade;
- bloqueios excessivos;
- consultas potencialmente maliciosas.

Esses eventos poderão ser integrados às ferramentas de observabilidade definidas na arquitetura da plataforma.

9.12 Hardening do PostgreSQL

O ambiente deverá seguir práticas de endurecimento (*hardening*).

Principais recomendações:

- desabilitar funcionalidades não utilizadas;
- restringir conexões externas;
- manter portas protegidas por firewall;
- remover usuários padrão desnecessários;
- limitar acesso administrativo;
- aplicar atualizações de segurança regularmente;
- revisar parâmetros de configuração.

Essas medidas reduzem a superfície de ataque do banco de dados.

9.13 Segurança em Ambientes Docker

Quando executado em contêineres, o PostgreSQL deverá observar:

- uso de redes privadas Docker;
- volumes persistentes protegidos;
- imagens oficiais e atualizadas;
- variáveis sensíveis fora do arquivo de imagem;

- restrição de exposição de portas ao ambiente externo.

Essas práticas complementam a segurança da infraestrutura containerizada.

9.14 Backup Seguro

Os backups deverão seguir políticas específicas de proteção.

Requisitos:

- armazenamento seguro;
- controle de acesso;
- criptografia quando necessário;
- retenção conforme política institucional;
- testes periódicos de restauração.

Backups devem receber o mesmo nível de proteção aplicado ao banco de dados principal.

9.15 Conformidade com a LGPD

O banco de dados deverá apoiar o cumprimento da Lei Geral de Proteção de Dados (LGPD).

Diretrizes:

- armazenar apenas os dados necessários;
- restringir acesso a informações pessoais;
- manter registros de auditoria;
- possibilitar anonimização quando aplicável;
- respeitar políticas de retenção e descarte de dados.

Essas medidas contribuem para a proteção da privacidade dos usuários e demais titulares de dados.

9.16 Resposta a Incidentes

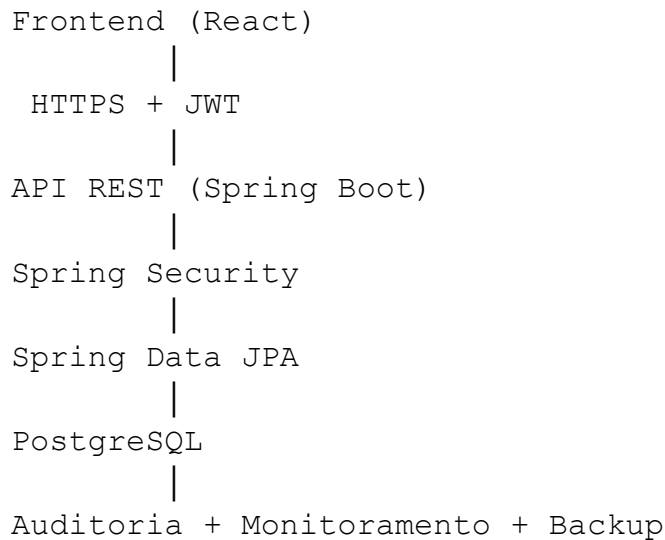
Em caso de incidente de segurança, deverão ser adotadas as seguintes etapas:

1. identificar o incidente;
2. isolar o ambiente afetado;
3. preservar evidências;
4. analisar a causa;
5. restaurar os serviços;
6. revisar controles para evitar recorrência.

Todo incidente deverá ser registrado e documentado.

9.17 Integração com os Demais Componentes

A segurança do banco integra-se aos demais elementos da arquitetura do GTP.



Essa integração garante proteção em todas as camadas da aplicação.

9.18 Boas Práticas

Durante a administração do banco deverão ser observadas as seguintes recomendações:

- utilizar usuários distintos para cada finalidade;
- aplicar o princípio do menor privilégio;
- proteger credenciais com variáveis de ambiente ou gerenciadores de segredos;
- habilitar criptografia em trânsito;

- manter backups protegidos e testados;
- registrar operações críticas em auditoria;
- monitorar continuamente o ambiente;
- manter PostgreSQL e dependências atualizados;
- revisar permissões e configurações periodicamente.

9.19 Relação com os Demais Documentos

Documento	Relação
Documento 06 – Arquitetura Backend	Implementa a integração segura com o PostgreSQL por meio do Spring Data JPA.
Documento 08 – Segurança	Define a política geral de segurança da aplicação.
Documento 09 – Docker	Estabelece a infraestrutura segura para execução do PostgreSQL.
Documento 10 – API REST	Consome os dados protegidos por esta arquitetura de segurança.
Documento 12 – Banco de Dados PostgreSQL	Complementa a administração e operação do banco de dados.

9.20 Conclusão do Capítulo

A segurança do banco de dados é um dos pilares fundamentais da arquitetura do GTP. A combinação de autenticação robusta, controle rigoroso de permissões, criptografia, auditoria, monitoramento contínuo e práticas de *hardening* garante que as informações armazenadas permaneçam protegidas contra acessos não autorizados, alterações indevidas e perda de dados.

Ao integrar essas medidas com as demais camadas da aplicação, o GTP estabelece uma arquitetura de persistência alinhada às boas práticas modernas de segurança, preparada para evoluir com novas demandas tecnológicas e regulatórias.

10 – Boas Práticas, Evolução do Banco de Dados e Conclusão

10.1 Objetivo

Este capítulo consolida as diretrizes para manutenção, evolução e operação do banco de dados PostgreSQL do GTP.

São apresentados:

- princípios gerais de desenvolvimento;
- boas práticas de modelagem;
- recomendações de desempenho;
- diretrizes de segurança;
- estratégia de evolução do esquema;
- integração com a arquitetura da aplicação;
- visão de longo prazo para crescimento do banco de dados.

O objetivo é garantir que o banco permaneça consistente, escalável, seguro e preparado para futuras versões do sistema.

10.2 Princípios Gerais

O banco de dados do GTP foi projetado considerando os seguintes princípios arquiteturais:

- simplicidade da modelagem;
- consistência dos dados;
- separação clara de responsabilidades;
- alta manutenibilidade;
- evolução incremental;
- compatibilidade com Spring Boot e Hibernate;
- versionamento completo do esquema;
- escalabilidade para novas funcionalidades.

Esses princípios orientam todas as decisões relacionadas ao projeto e à evolução da camada de persistência.

10.3 Boas Práticas de Modelagem

Durante a criação de novas tabelas deverão ser observadas as seguintes recomendações:

- utilizar nomes claros e padronizados;
- adotar snake_case para tabelas e colunas;
- definir chaves primárias numéricas (BIGINT);
- evitar redundância de dados;
- normalizar o modelo até, no mínimo, a Terceira Forma Normal (3FN), avaliando desnormalizações apenas quando justificadas por desempenho;
- documentar tabelas e colunas com COMMENT ON;
- utilizar restrições para garantir integridade.

A modelagem deverá sempre refletir o domínio de negócio definido no Documento 11 – Modelo Entidade-Relacionamento.

10.4 Boas Práticas de Desenvolvimento

A implementação da camada de persistência deverá seguir as seguintes diretrizes:

- utilizar Flyway para toda alteração estrutural;
- manter entidades JPA sincronizadas com o esquema;
- evitar consultas SQL duplicadas;
- priorizar Spring Data JPA para operações comuns;
- utilizar consultas nativas apenas quando houver ganho comprovado de desempenho ou necessidade específica;
- versionar todas as alterações no repositório Git;
- revisar impacto das mudanças antes da implantação.

10.5 Boas Práticas de Desempenho

Para manter o banco eficiente ao longo do tempo, recomenda-se:

- criar índices apenas quando necessários;
- monitorar consultas lentas;
- revisar planos de execução (EXPLAIN ANALYZE);
- atualizar estatísticas periodicamente (ANALYZE);
- executar rotinas de manutenção (VACUUM);
- evitar consultas com SELECT * em produção;
- limitar resultados por meio de paginação;
- utilizar projeções (DTOs) para reduzir a transferência de dados quando apropriado.

O desempenho deverá ser continuamente acompanhado por métricas e ferramentas de observabilidade.

10.6 Boas Práticas de Segurança

As recomendações apresentadas no Documento 08 – Segurança e no Capítulo 9 deste documento deverão ser observadas permanentemente.

Em especial:

- utilizar usuários distintos por função;
- aplicar o princípio do menor privilégio;
- proteger credenciais em variáveis de ambiente ou gerenciadores de segredos;
- habilitar TLS/SSL em ambientes distribuídos;
- registrar operações críticas em auditoria;
- manter o PostgreSQL atualizado;
- revisar permissões regularmente;
- proteger backups e arquivos de configuração.

10.7 Evolução do Modelo de Dados

O banco foi concebido para evoluir de forma incremental.

Novas funcionalidades poderão incluir:

- novos módulos funcionais;
- tabelas adicionais;
- relacionamentos complementares;
- campos opcionais;
- novos índices;
- views e materialized views;
- funções e procedimentos auxiliares.

Toda evolução deverá preservar a compatibilidade com versões anteriores, sempre que possível.

10.8 Estratégia de Versionamento

A evolução do banco seguirá os princípios estabelecidos para o Flyway.

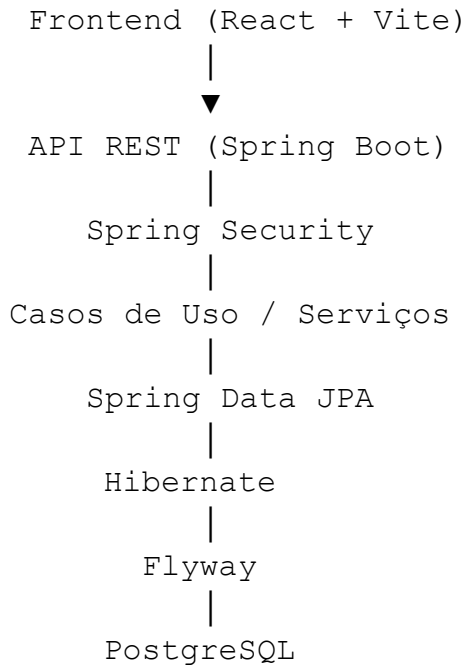
Cada alteração deverá:

- possuir uma migração própria;
- ser numerada sequencialmente;
- conter descrição clara;
- ser testada antes da implantação;
- ser aplicada de forma consistente em todos os ambientes.

Alterações em scripts já executados não serão permitidas em ambientes compartilhados.

10.9 Integração com a Arquitetura do Sistema

O banco de dados integra-se às demais camadas do GTP conforme a arquitetura definida nos documentos técnicos.



Essa organização garante separação de responsabilidades, facilidade de testes e manutenção.

10.10 Escalabilidade

A arquitetura do banco permite crescimento progressivo.

Entre as possibilidades futuras estão:

- particionamento de tabelas de grande volume;
- replicação para leitura;
- balanceamento de carga;
- cache distribuído;
- armazenamento de documentos em sistemas especializados, quando adequado;
- integração com mecanismos de busca para pesquisas textuais avançadas.

Essas estratégias serão adotadas conforme a evolução do volume de dados e das necessidades operacionais.

10.11 Monitoramento Contínuo

O ambiente deverá ser monitorado permanentemente.

Aspectos acompanhados:

- crescimento do banco;
- utilização de CPU e memória;
- consultas lentas;
- uso de índices;
- conexões ativas;
- espaço em disco;
- bloqueios;
- falhas de integridade;
- sucesso das rotinas de backup.

As métricas subsidiarão decisões de otimização e expansão da infraestrutura.

10.12 Documentação e Governança

Toda alteração no banco deverá ser refletida na documentação oficial do projeto.

Os seguintes artefatos deverão permanecer sincronizados:

- Documento 06 – Arquitetura Backend;
- Documento 08 – Segurança;
- Documento 09 – Docker;
- Documento 10 – API REST;
- Documento 11 – Modelo Entidade-Relacionamento (DER);
- Documento 12 – Banco de Dados PostgreSQL.

Essa governança garante rastreabilidade e reduz inconsistências entre documentação e implementação.

10.13 Relação com os Demais Documentos

Documento	Relação
Documento 06 – Arquitetura Backend	Define a camada de persistência e integração com JPA/Hibernate.
Documento 08 – Segurança	Estabelece as políticas gerais de proteção aplicadas ao banco de dados.
Documento 09 – Docker	Disponibiliza o ambiente PostgreSQL em contêineres.
Documento 10 – API REST	Consome os dados persistidos e expõe os recursos da aplicação.
Documento 11 – Modelo Entidade-Relacionamento	Serve de base para a implementação física do banco.

10.14 Lições Arquiteturais

Ao longo da elaboração deste documento, foram adotadas algumas decisões estratégicas:

- utilização do PostgreSQL como SGBD principal;
- Flyway como mecanismo exclusivo de evolução do esquema;
- Spring Data JPA e Hibernate para mapeamento objeto-relacional;
- separação entre regras de negócio e responsabilidades do banco;
- priorização de padrões amplamente adotados pela comunidade Java;
- documentação completa como parte integrante do processo de desenvolvimento.

Essas decisões favorecem a sustentabilidade do projeto e facilitam sua manutenção por diferentes equipes.

10.15 Perspectivas Futuras

Entre as evoluções previstas para versões futuras do GTP destacam-se:

- otimização avançada de consultas;

- expansão dos módulos funcionais;
- indicadores analíticos mais sofisticados;
- integração com ferramentas de Business Intelligence (BI);
- replicação para alta disponibilidade;
- automação de rotinas administrativas;
- monitoramento inteligente baseado em métricas;
- adoção de recursos avançados do PostgreSQL, conforme necessidade.

A arquitetura atual foi planejada para suportar essas evoluções sem necessidade de reestruturações significativas.

Conclusão Geral

O **Documento 12 – Banco de Dados PostgreSQL** estabelece a especificação técnica completa da camada de persistência do **Gestor de Territórios e Publicações (GTP)**. Ao longo dos capítulos foram definidos:

- a arquitetura geral do banco de dados;
- a estrutura física das tabelas;
- os tipos de dados e convenções;
- os mecanismos de integridade e indexação;
- os objetos auxiliares do PostgreSQL;
- a estratégia de versionamento com Flyway;
- as práticas de administração e monitoramento;
- as políticas de segurança;
- as diretrizes para evolução contínua do esquema.

Em conjunto, essas definições fornecem uma base sólida para a implementação do banco de dados, assegurando consistência, desempenho, segurança e alinhamento com a arquitetura em camadas adotada pelo GTP.

A padronização estabelecida neste documento favorece o desenvolvimento colaborativo, reduz riscos durante a evolução do sistema e garante que a camada de persistência acompanhe o crescimento funcional da plataforma de forma controlada, rastreável e sustentável.